CE-1103 · Lección 05

Estructuras de Datos Jerárquicas

Organizar los datos en forma de árbol

TreeNode.java

```
class TreeNode {
    int key;
    TreeNode left;
    TreeNode right;
}
```

Instituto Tecnológico de Costa Rica · Ingeniería en Computadores

// introducción

¿Qué aprenderemos hoy?

1. Identificar la terminología de los árboles y sus recorridos
2. Implementar un BST: búsqueda, inserción y eliminación
3. Representar un heap con un arreglo
4. Mantener el balance con árboles AVL y rotaciones
5. Construir árboles de expresión a partir de una expresión
6. Describir los árboles B y los tries

// introducción

Familias de árboles

Árboles binarios

Cada nodo tiene a lo sumo dos hijos.

- **BST**: búsquedas eficientes
- **AVL** y **rojinegros**: balanceados
- **Splay**: lo más usado, cerca de la raíz
- **Expresiones**: expresiones matemáticas
- **Heap**: colas de prioridad

vs

Árboles N-arios

Cada nodo tiene un número variable de hijos.

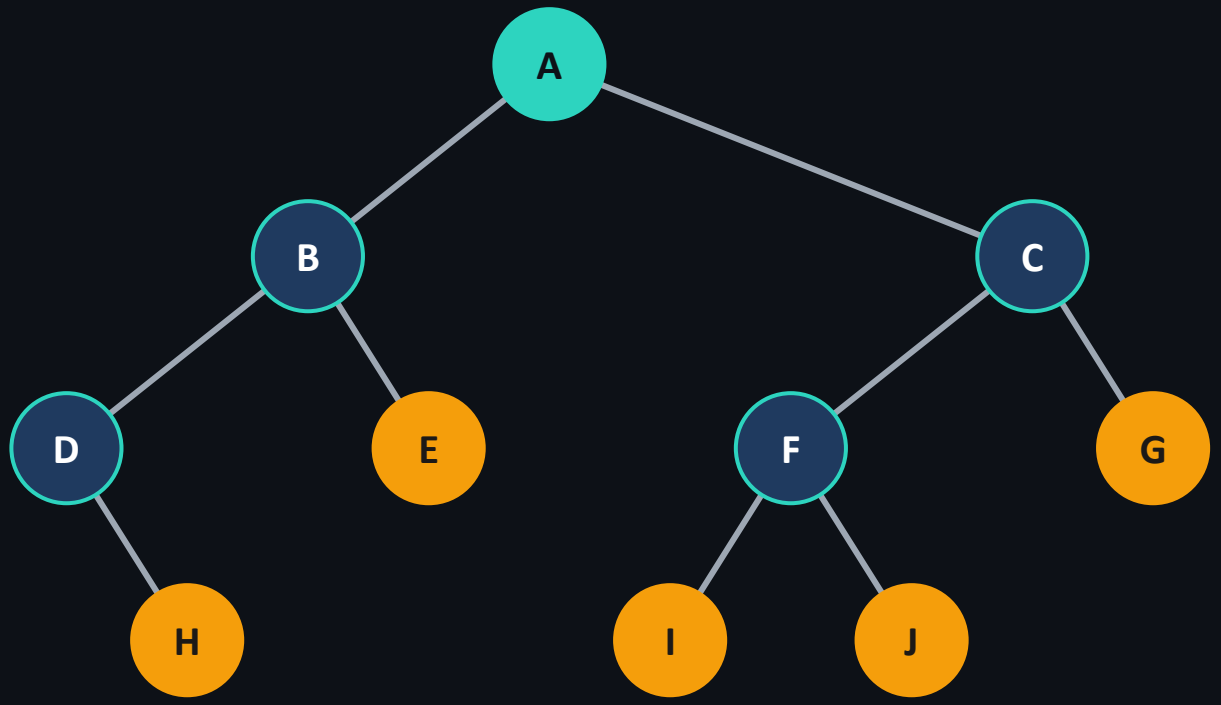
- **B** y **B+**: datos en disco, bases de datos
- **Trie**: conjuntos de cadenas de caracteres
- **Sufijos**: todas las subcadenas de una cadena

En esta lección: BST, heap, AVL, expresiones, árboles B y tries. Casi todos se implementan con **nodos enlazados**, como las listas.

// terminología

Terminología básica de los árboles

Raíz | Nodo interno | Hoja



Término	En el ejemplo
Raíz	A, en el nivel 0
Padre e hijo	B es el padre de D y E
Hijo izquierdo	D es el hijo izquierdo de B
Hoja	Nodo sin hijos: H, E, I, J, G
Nivel	A: 0 · B y C: 1 · D, E, F y G: 2 · H, I y J: 3
Subárbol	F con sus hijos I y J
Camino	A → C → F → J

// terminología

Tipo de dato abstracto Árbol

1

Insertar (insert)

Añade un nodo al árbol.

2

Eliminar (delete)

Elimina un nodo del árbol.

3

Buscar (search)

Busca un nodo en el árbol.

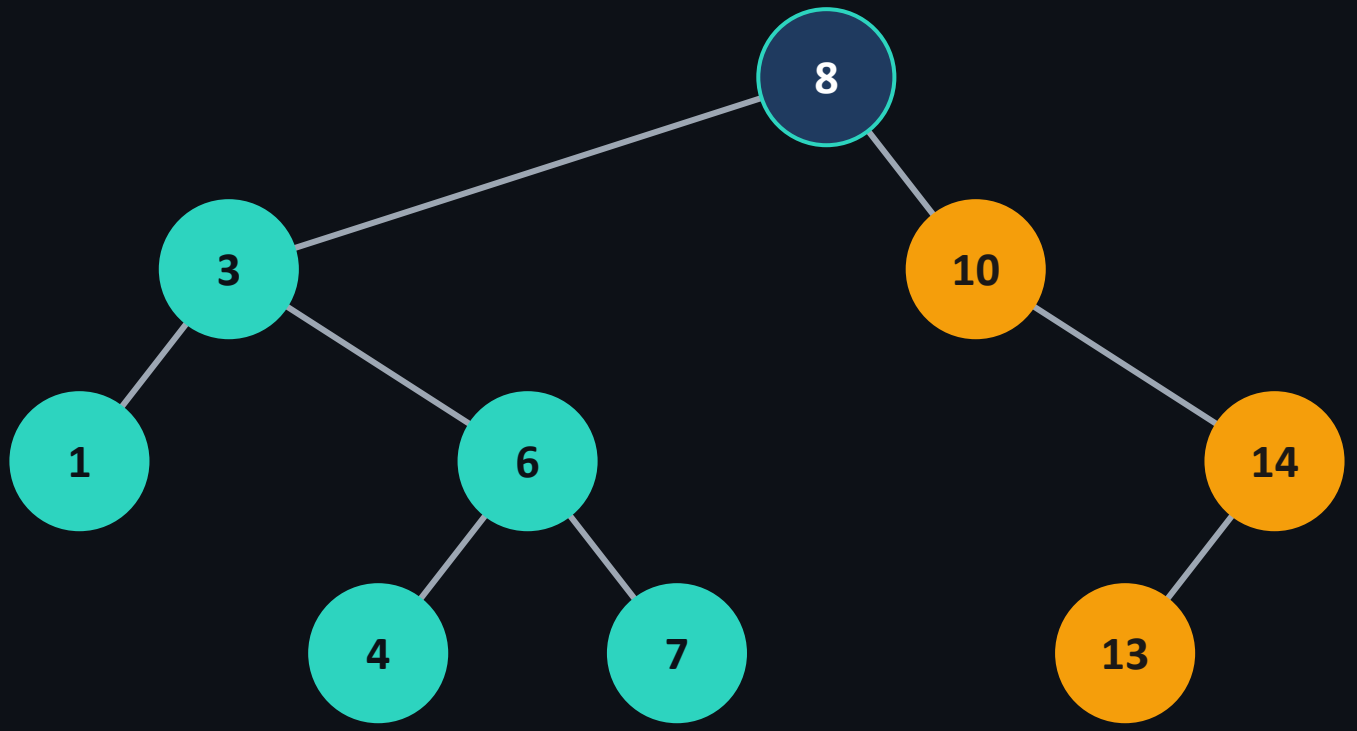
4

Recorrer (traverse)

Visita todos los nodos en un orden definido.

Recorrer admite varios órdenes: **preorden**, **inorden**, **postorden** y **por niveles**.

Árbol binario de búsqueda (BST)



Subárbol izquierdo

Todos los valores son **menores** que 8.

Subárbol derecho

Todos los valores son **mayores** que 8.

Propiedad: para cada nodo, **izquierdo < nodo < derecho**. Si la llave ya existe, no se inserta.

// árboles binarios de búsqueda

Implementación con nodos enlazados

Nodo del árbol

```
class TreeNode {
    int key;
    TreeNode left, right;

    public TreeNode(int item) {
        key = item;
        left = right = null;
    }
}
```

El árbol

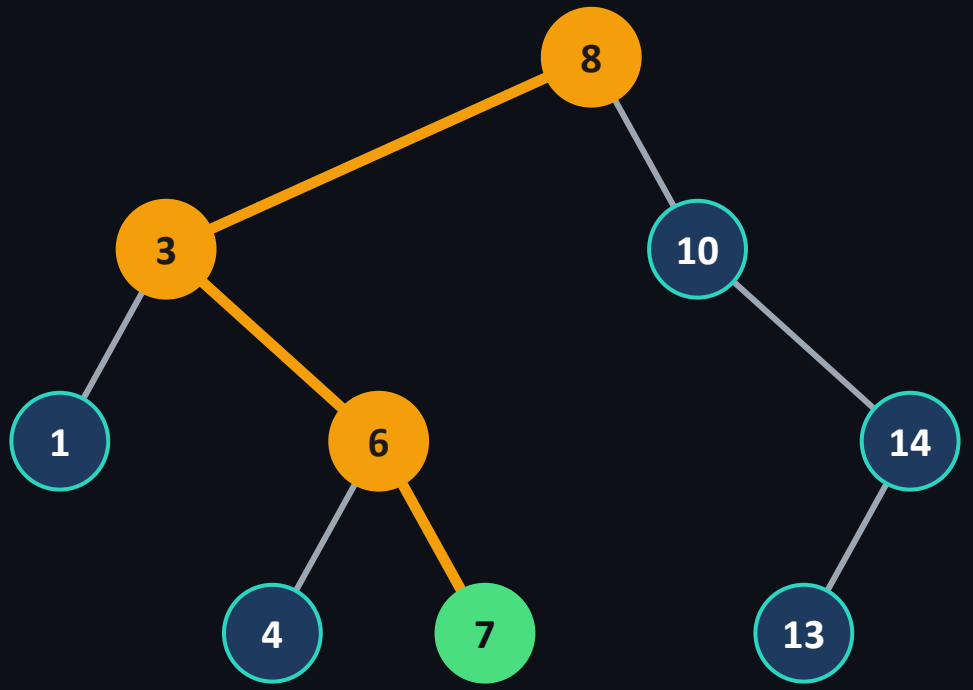
```
public class BinarySearchTree {
    TreeNode root;

    public BinarySearchTree() {
        root = null;
    }
}
```

Nodos enlazados

Cada nodo guarda su llave y las referencias a sus hijos; un árbol vacío tiene `root = null`.

Búsqueda: encontrar el 7



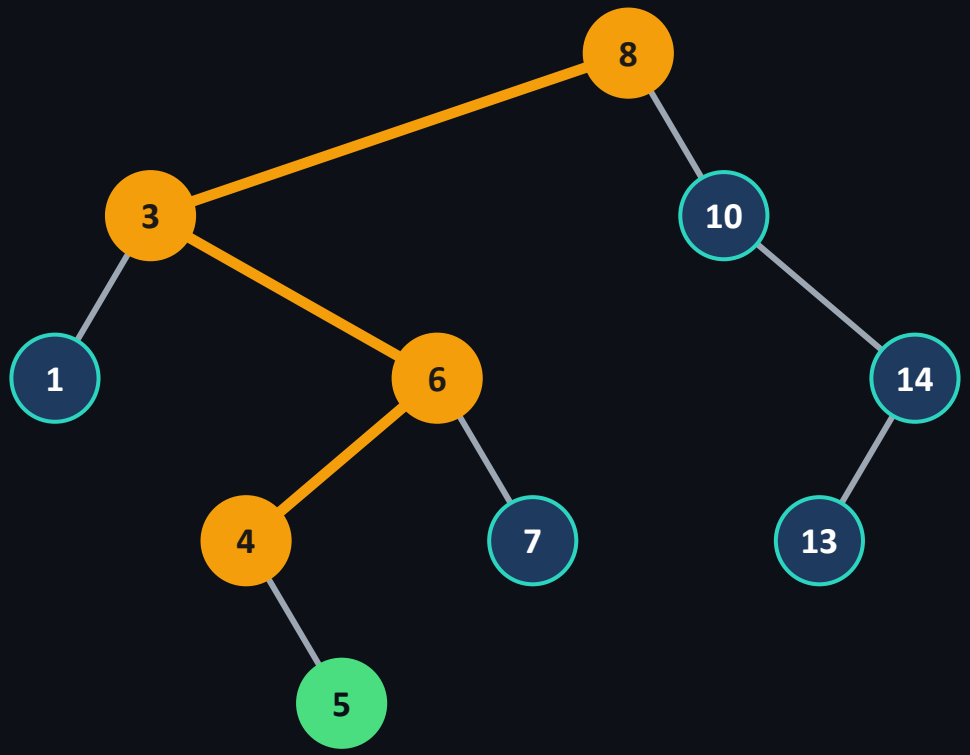
Búsqueda recursiva

```
private boolean searchRecursive(TreeNode root, int key) {
    if (root == null) return false;
    if (root.key == key) return true;
    if (key > root.key)
        return searchRecursive(root.right, key);
    return searchRecursive(root.left, key);
}
```

Pareja de métodos: `search` (público) recibe los parámetros iniciales; `searchRecursive` (privado) hace la operación recursiva.

Recorrido: 8 → 3 → 6 → 7. En cada nodo se descarta un subárbol completo: 7 < 8 va a la izquierda, 7 > 3 y 7 > 6 van a la derecha.

Inserción: agregar el 5



Inserción recursiva

```
public void insert(int key) {
    root = insertRecursive(root, key);
}
private TreeNode insertRecursive(TreeNode root, int key) {
    if (root == null) return new TreeNode(key);
    if (key < root.key)
        root.left = insertRecursive(root.left, key);
    else if (key > root.key)
        root.right = insertRecursive(root.right, key);
    return root;
}
```

Se sigue la misma ruta que la búsqueda (8 → 3 → 6 → 4); el nodo nuevo queda como **hoja**, a la derecha del 4.

Error de principiante al insertar

Incorrecto

```
// insertRecursive devuelve void
insertRecursive(root, key);

// dentro del método:
root = new TreeNode(key);
return;
```

El cambio se pierde: `root` es una copia de la referencia.

vs

Correcto

```
// insertRecursive devuelve TreeNode
root = insertRecursive(root, key);

// dentro del método:
return new TreeNode(key);
```

Se reasigna el resultado: el árbol sí se modifica.

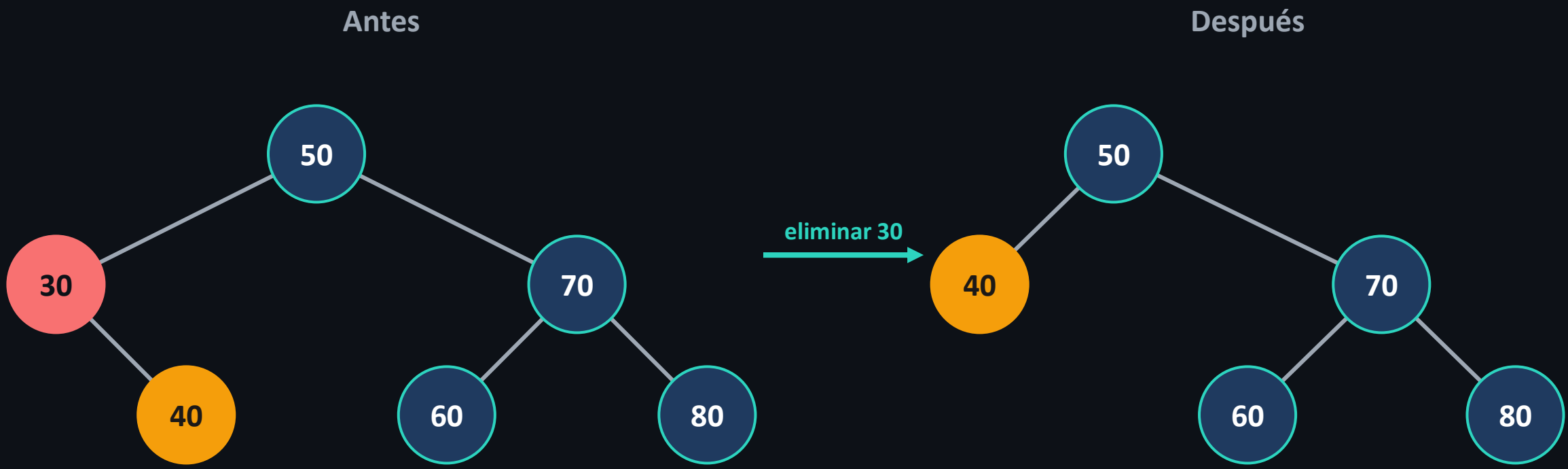
Ojo: las referencias en Java se pasan **por valor**; asignar `root` dentro del método no cambia el `root` de quien llama.

Eliminación: nodo hoja



Hoja: no tiene hijos, así que **simplemente se elimina**.

Eliminación: nodo con un solo hijo



Un hijo: el hijo (40) **reemplaza** al nodo eliminado (30) y queda enlazado a su padre (50).

Eliminación: nodo con dos hijos



Un solo criterio: el 50 se reemplaza por el **menor del subárbol derecho** (60), que luego se elimina de su lugar. También valdría el mayor del izquierdo, pero use solo una estrategia.

Código: eliminar un nodo

deleteRecursive

```
private TreeNode deleteRecursive(TreeNode root, int key) {
    if (root == null) return root;

    if (key < root.key)
        root.left = deleteRecursive(root.left, key);
    else if (key > root.key)
        root.right = deleteRecursive(root.right, key);
    else {
        if (root.left == null) return root.right;
        else if (root.right == null) return root.left;

        root.key = minValue(root.right);
        root.right = deleteRecursive(root.right, root.key);
    }
    return root;
}
```

minValue

```
int minValue(TreeNode root) {
    while (root.left != null)
        root = root.left;
    return root.key;
}
```

Dos hijos

Se copia el menor del subárbol derecho y se elimina ese nodo.

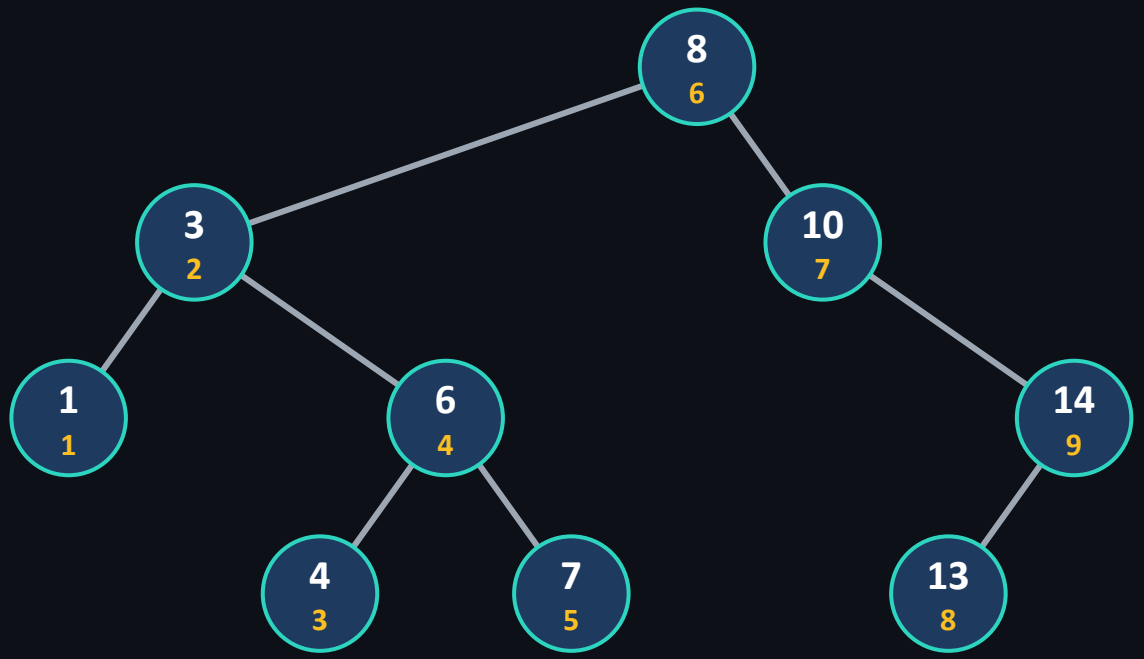
Cuatro recorridos de un mismo árbol

Recorrido	Orden de visita	Resultado en el BST de ejemplo
Preorden	raíz → izquierdo → derecho	8 3 1 6 4 7 10 14 13
Inorden	izquierdo → raíz → derecho	1 3 4 6 7 8 10 13 14
Postorden	izquierdo → derecho → raíz	1 4 7 6 3 13 14 10 8
Por niveles	un nivel completo antes del siguiente	8 3 10 1 6 14 4 7 13

Recorrer un árbol no solo sirve para imprimirlo: permite **operar sobre todos los nodos**.

Recorrido inorden: izquierda, raíz, derecha

Número en el nodo = orden de visita



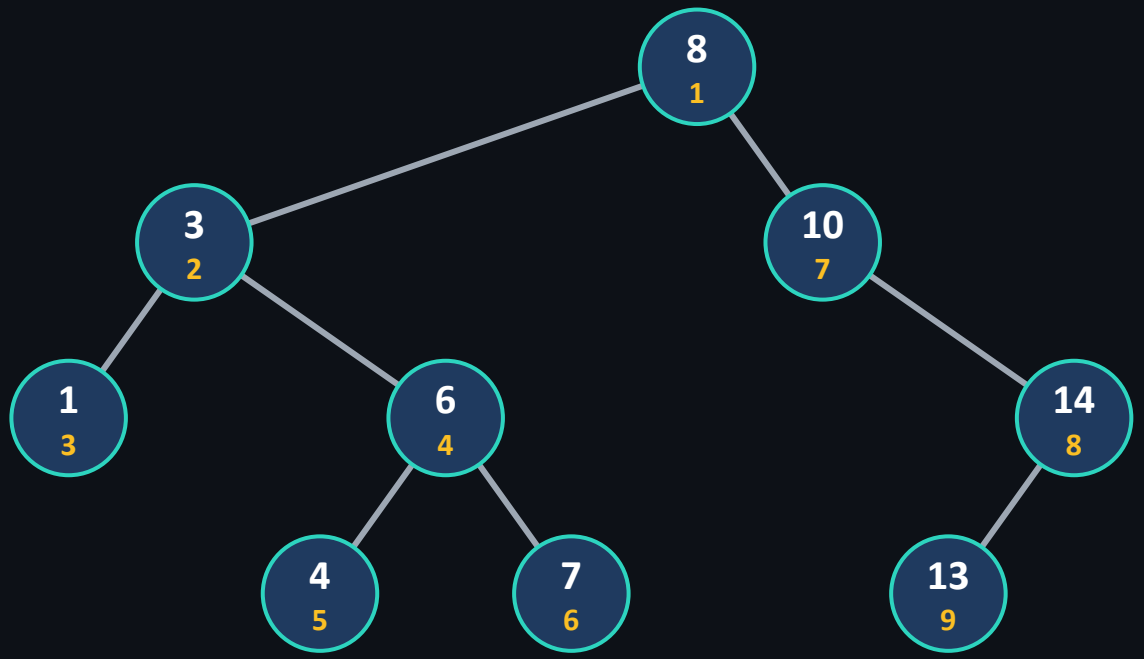
inOrder

```
private void inOrderRecursive(TreeNode root) {
    if (root != null) {
        inOrderRecursive(root.left);
        System.out.print(root.key + " ");
        inOrderRecursive(root.right);
    }
}
```

Se imprime: `1 3 4 6 7 8 10 13 14`. En un BST el inorden da los valores en orden **ascendente**.

Recorrido preorden: raíz, izquierda, derecha

Número en el nodo = orden de visita



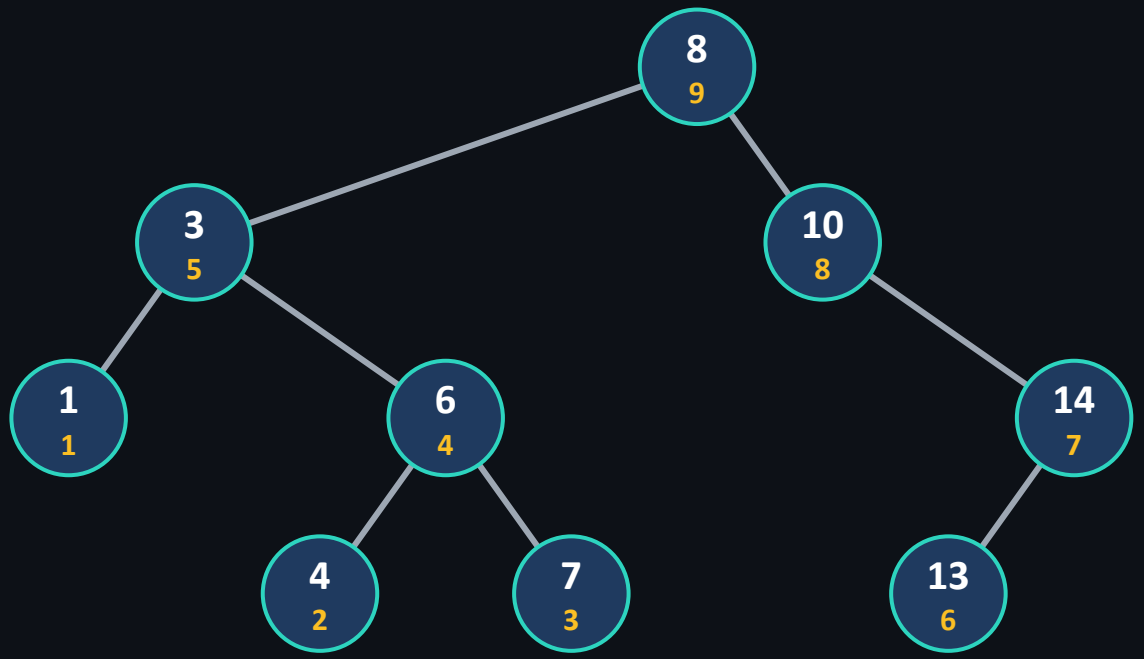
preOrder

```
private void preOrderRecursive(TreeNode root) {
    if (root != null) {
        System.out.print(root.key + " ");
        preOrderRecursive(root.left);
        preOrderRecursive(root.right);
    }
}
```

Se imprime: `8 3 1 6 4 7 10 14 13`. La raíz se visita **antes** que sus subárboles.

Recorrido postorden: izquierda, derecha, raíz

Número en el nodo = orden de visita



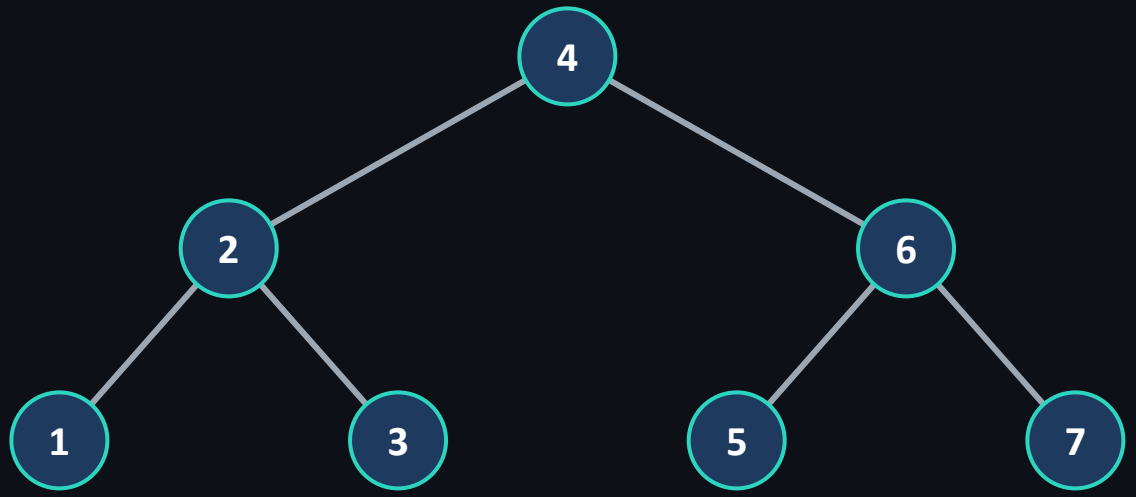
postOrder

```
private void postOrderRecursive(TreeNode root) {
    if (root != null) {
        postOrderRecursive(root.left);
        postOrderRecursive(root.right);
        System.out.print(root.key + " ");
    }
}
```

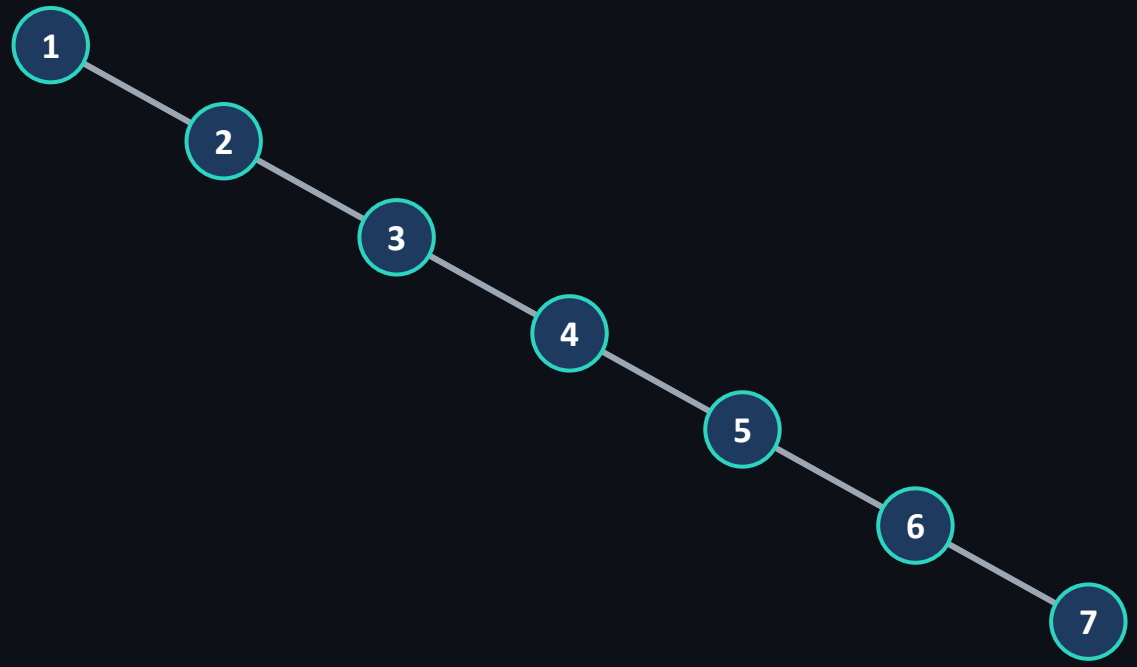
Se imprime: `1 4 7 6 3 13 14 10 8`. La raíz se visita **después** de sus subárboles.

El problema: un BST puede degenerar

Orden 4, 2, 6, 1, 3, 5, 7 · altura 3



Orden 1, 2, 3, 4, 5, 6, 7 · altura 7



Problema: con inserciones en orden el BST se vuelve una **lista enlazada**: buscar, insertar y eliminar pasan a tiempo **lineal**.

Heap (montículo)

1

Árbol completo

Todos los niveles están llenos, salvo quizá el último, que se llena de izquierda a derecha.

2

Propiedad de heap

Heap máximo: cada nodo ≥ sus hijos. **Heap mínimo:** cada nodo ≤ sus hijos.

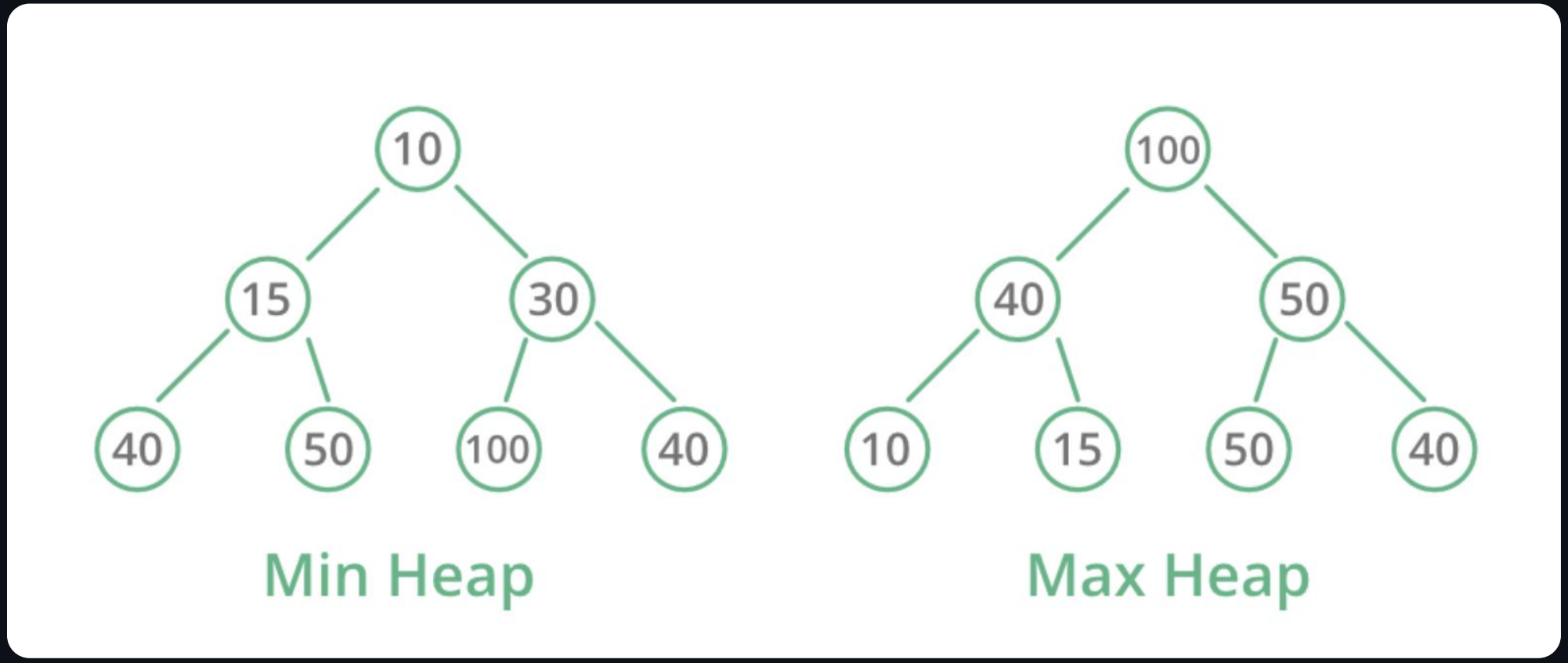
3

Para qué sirve

Es la opción natural para las colas de prioridad y la base de heap sort.

A diferencia del BST, el heap se implementa sobre un **arreglo**, no con nodos enlazados.

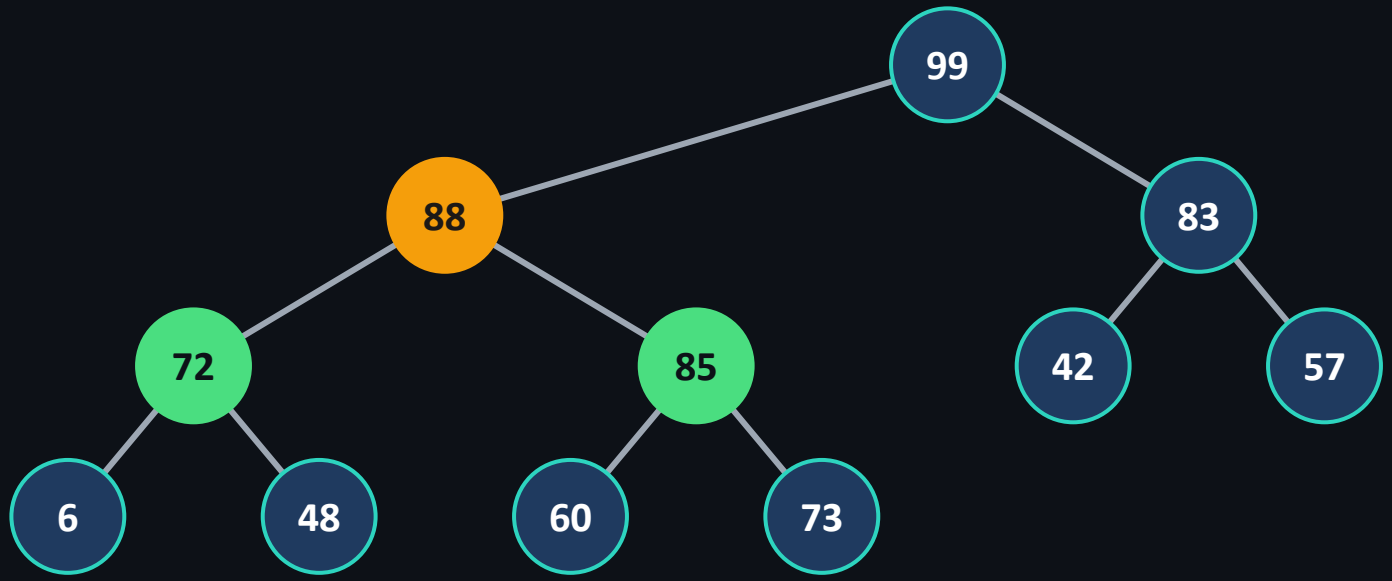
Heap mínimo y heap máximo



Min heap (izquierda): cada nodo ≤ sus hijos · Max heap (derecha): cada nodo ≥ sus hijos

Heap en un arreglo

99	88	83	72	85	42	57	6	48	60	73
0	1	2	3	4	5	6	7	8	9	10



Posición	Fórmula
Raíz	posición 0
Hijo izquierdo de i	2i + 1
Hijo derecho de i	2i + 2
Padre de i	(i − 1) / 2

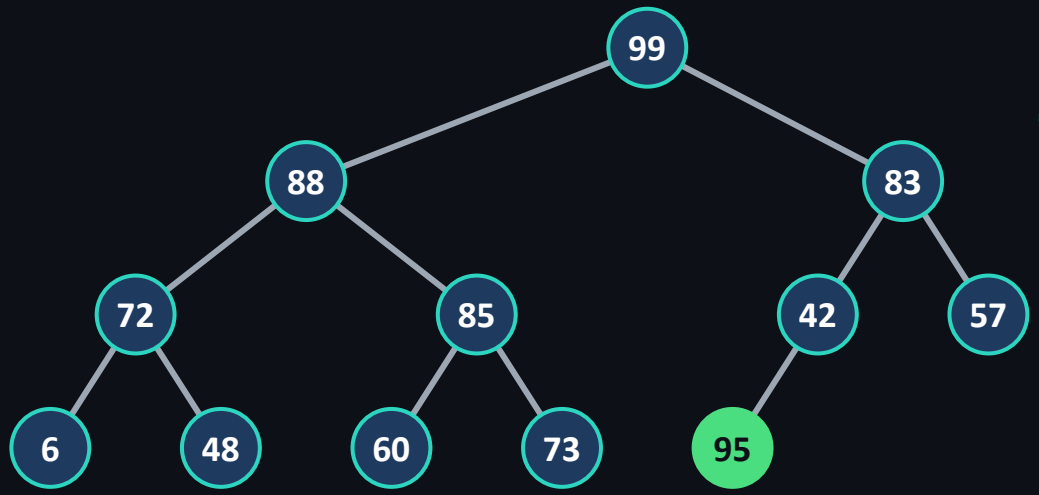
Como el árbol es completo, se guarda sin huecos: cabe en un arreglo.

Para `i = 1` (valor 88): los hijos están en `2·1+1 = 3` y `2·1+2 = 4`, es decir, 72 y 85.

Insertar el 95 en un heap máximo

1. Se añade al final del arreglo

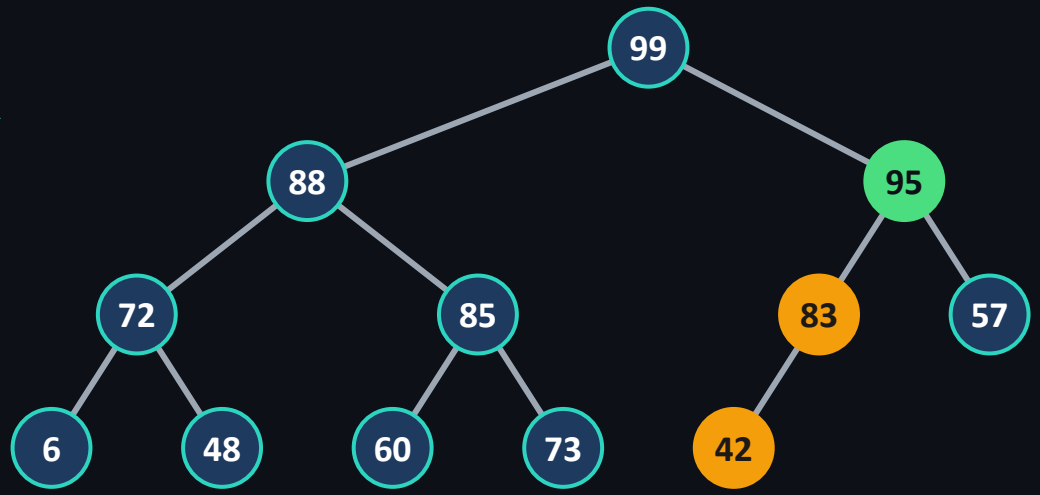
0	1	2	3	4	5	6	7	8	9	10	11
99	88	83	72	85	42	57	6	48	60	73	95



subir el 95

2. Sube mientras sea mayor que su padre

0	1	2	3	4	5	6	7	8	9	10	11
99	88	95	72	85	83	57	6	48	60	73	42

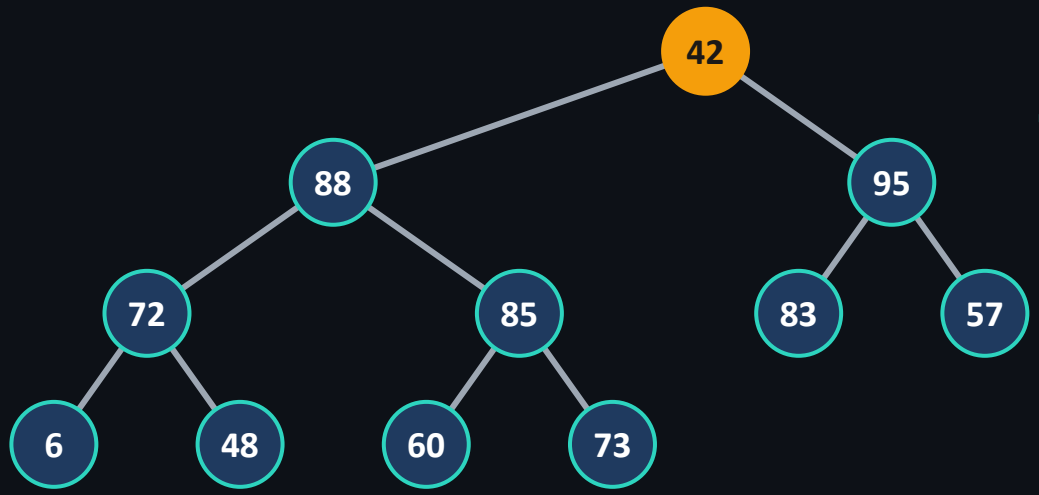


95 > 42: se intercambian · 95 > 83: se intercambian · 95 < 99: se detiene. El heap sigue cumpliendo su propiedad.

Eliminar la raíz de un heap máximo

1. El último elemento ocupa la raíz

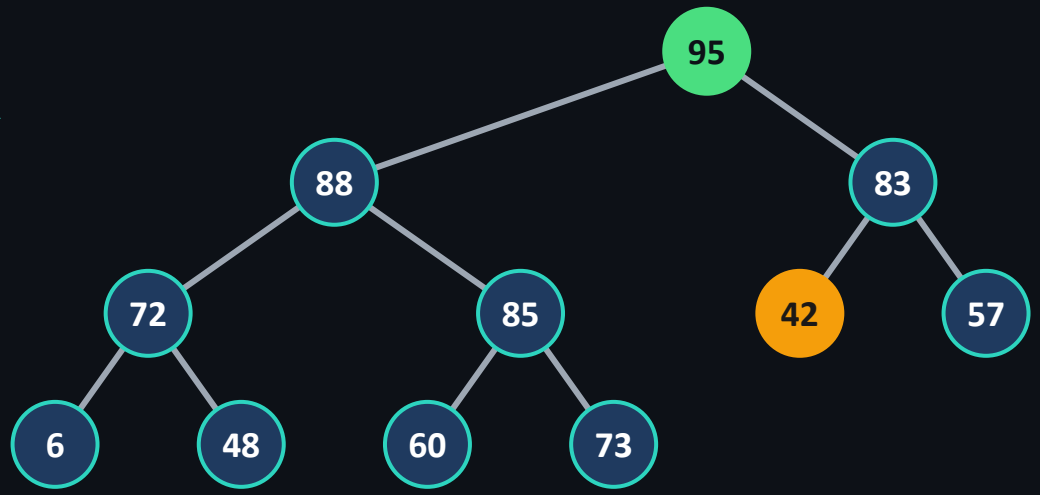
42	88	95	72	85	83	57	6	48	60	73
0	1	2	3	4	5	6	7	8	9	10



heapifyDown

2. Baja con el hijo mayor

95	88	83	72	85	42	57	6	48	60	73
0	1	2	3	4	5	6	7	8	9	10



Sale el 99. El 42 baja: se intercambia con 95 (el hijo mayor) y luego con 83; en una hoja se detiene.

// heap

HeapTree: insert, delete y get

Estructura y get()

```
public class HeapTree {
    private int[] heap;
    private int size;

    public HeapTree(int capacity) {
        heap = new int[capacity];
        size = 0;
    }

    public int get() {
        return heap[0];
    }
}
```

El código completo lanza `IllegalStateException` si el heap está lleno (insert) o vacío (delete, get).

insert()

```
public void insert(int element) {
    heap[size++] = element;
    heapifyUp(size - 1);
}
```

delete()

```
public int delete() {
    int root = heap[0];
    heap[0] = heap[size - 1];
    size--;
    heapifyDown(0);
    return root;
}
```

// heap

Código: heapifyUp

Subir un elemento

```
private void heapifyUp(int index) {
    int parentIndex = (index - 1) / 2;

    while (index > 0 && heap[index] > heap[parentIndex]) {
        swap(index, parentIndex);
        index = parentIndex;
        parentIndex = (index - 1) / 2;
    }
}
```

Sube

Mientras el elemento sea mayor que su padre, los intercambia.

Se detiene

Al llegar a la raíz o cuando su padre ya es mayor.

Costo: O(log n)

A lo sumo la altura del árbol.

// heap

Código: heapifyDown

Bajar un elemento

```
private void heapifyDown(int index) {
    int left = 2 * index + 1;
    int right = 2 * index + 2;
    int largest = index;

    if (left < size && heap[left] > heap[largest])
        largest = left;
    if (right < size && heap[right] > heap[largest])
        largest = right;

    if (largest != index) {
        swap(index, largest);
        heapifyDown(largest);
    }
}
```

Elige el mayor

Compara el nodo con sus dos hijos y se queda con el mayor.

Intercambia y baja

Si el mayor no es el nodo, los intercambia y repite en ese hijo.

Costo: O(log n)

A lo sumo la altura del árbol.

// árboles avl

Árboles AVL: BST balanceados

Auto-balanceado

El primer árbol auto-balanceado (Adelson-Velsky y Landis, 1962).

Es un BST

Un árbol binario de búsqueda balanceado en altura.

Factor de balance

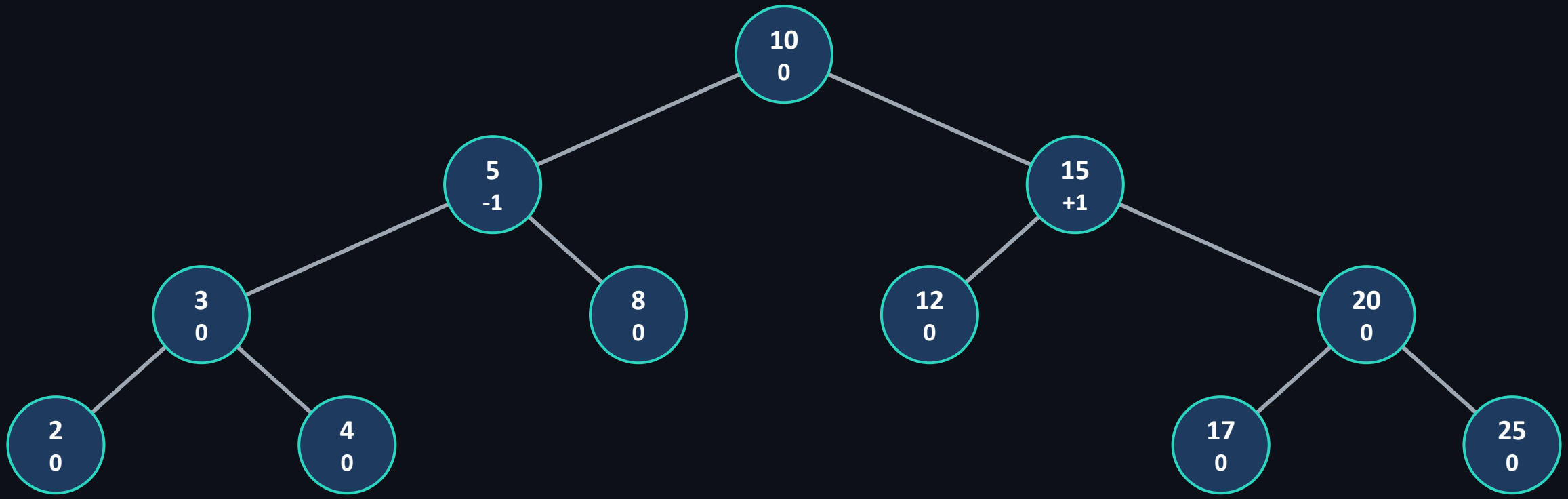
En cada nodo vale −1, 0 o 1.

Tiempo garantizado

Buscar, insertar y eliminar: **O(log n)**.

Se rebalancea con **rotaciones** después de insertar o eliminar un nodo.

Factor de balance de cada nodo



Factor de balance = altura del subárbol derecho − altura del izquierdo. Aquí todos valen −1, 0 o 1: es un AVL.

Ventajas y desventajas del AVL

Ventajas

Al estar balanceado, la altura se mantiene en O(log n).

- Búsquedas rápidas
- Auto-balanceados

vs

Desventajas

- Requieren más memoria que un BST no balanceado
- Inserción y eliminación más lentas
- Las rotaciones pueden ser costosas
- Difíciles de implementar

Se paga un costo extra al modificar el árbol a cambio de búsquedas siempre rápidas.

Rebalancear: inserción y eliminación

Inserción del nodo w

- Inserción normal de BST para w
- De w hacia la raíz: **z** = primer nodo desbalanceado
- **y** = hijo de z en el camino; **x** = nieto de z en el camino
- Rotaciones simples o dobles en el subárbol de z

Eliminación del nodo w

- Eliminación normal de BST
- De w hacia arriba: **z** = primer nodo desbalanceado
- **y** = hijo de z de mayor altura; **x** = hijo de y de mayor altura
- Rotaciones apropiadas en el subárbol de z

> En ambos casos hay **4 disposiciones** posibles de x, y y z: LL, RR, LR y RL.

Casos simples: una rotación

LL (izquierda-izquierda): y y x son hijos izquierdos → rotación derecha



RR (derecha-derecha): y y x son hijos derechos → rotación izquierda

> En ambos casos **y** pasa a ser la raíz del subárbol y **z** baja como su hijo.

Casos dobles: dos rotaciones

LR: y es hijo izquierdo de z; x es hijo derecho de y

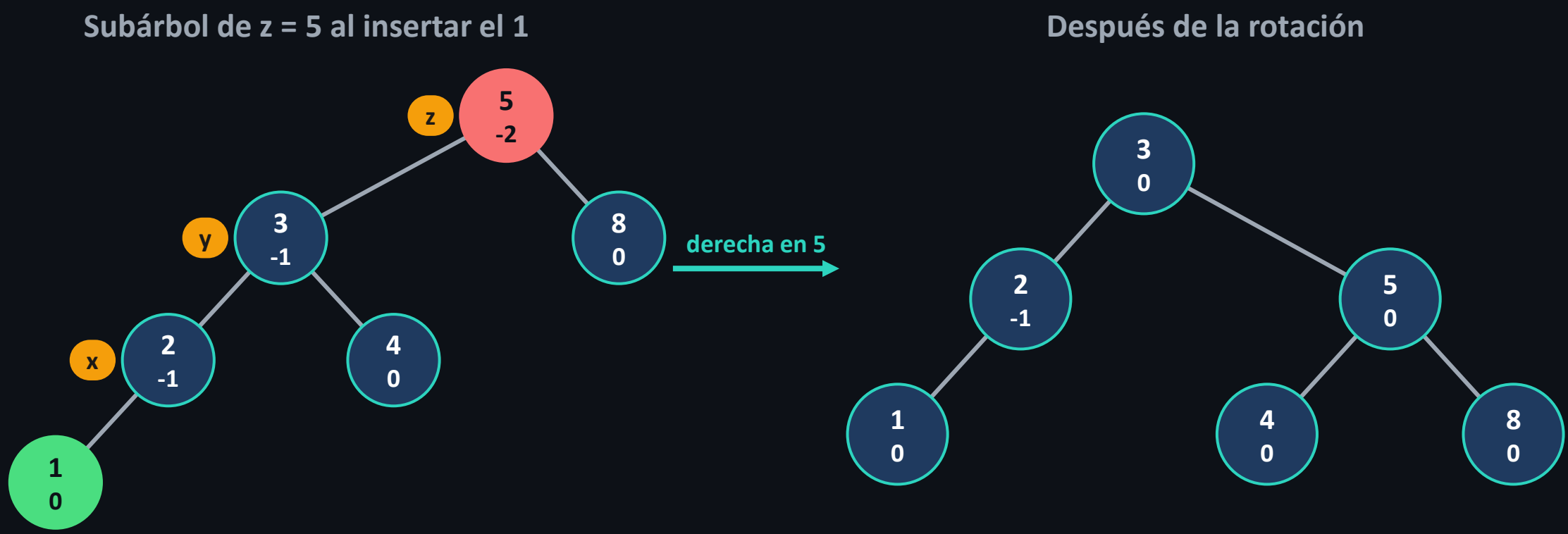


RL: y es hijo derecho de z; x es hijo izquierdo de y



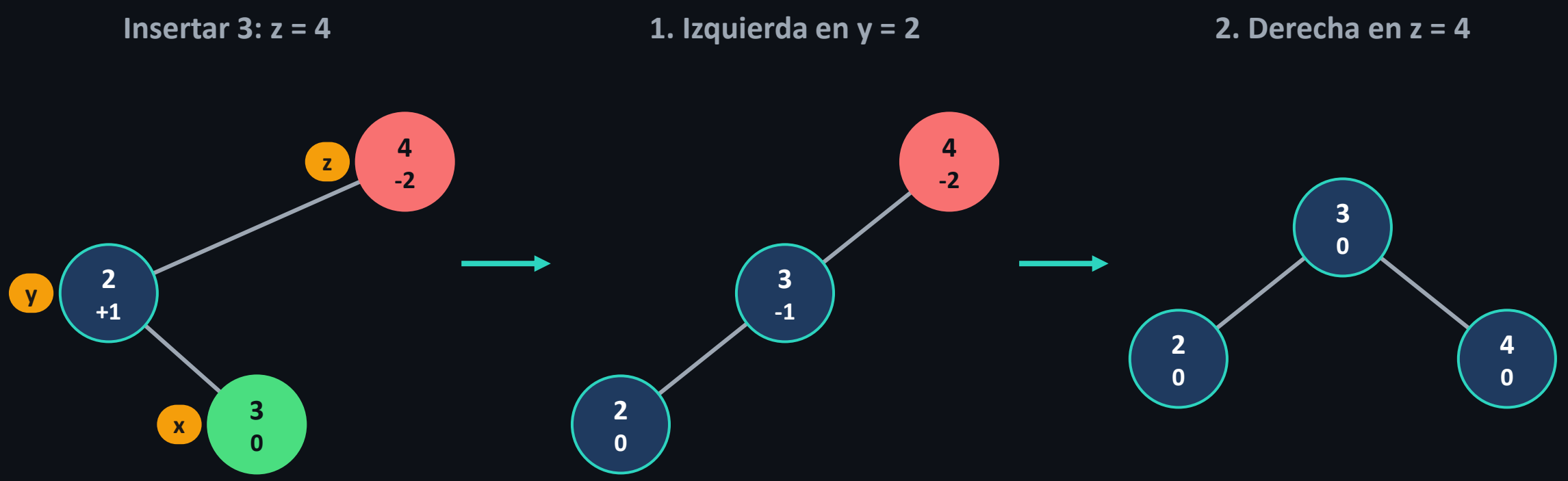
> La primera rotación convierte el caso en uno simple (LL o RR); la segunda lo resuelve.

Ejemplo: insertar 1 → rotación derecha



Se inserta el 1 en el AVL del ejemplo: el 5 queda con factor **−2**. Como y = 3 y x = 2 son hijos izquierdos: caso **LL**.

Ejemplo: insertar 3 → rotación izquierda-derecha



x = 3 es hijo derecho de **y** = 2, e **y** es hijo izquierdo de **z** = 4: caso **LR**, dos rotaciones.

Código: rotación a la derecha

rightRotate

```
private AVLNode rightRotate(AVLNode y) {
    AVLNode x = y.left;
    AVLNode T2 = x.right;

    // Rotación
    x.right = y;
    y.left = T2;

    // Actualizar alturas
    y.height = Math.max(height(y.left), height(y.right)) + 1;
    x.height = Math.max(height(x.left), height(x.right)) + 1;

    return x;  // nueva raíz del subárbol
}
```

leftRotate

Es la versión espejo: sube `x.right` y `x` pasa a ser su hijo izquierdo.

Alturas

Primero se recalcula el nodo que bajó y luego el que subió.

Retorno

La rotación devuelve la nueva raíz del subárbol.

Código: insertar en un AVL

```
private AVLNode insertRecursive(AVLNode node, int key) {
    // ... inserción normal de BST (sin duplicados)
    node.height = 1 + Math.max(height(node.left),
                               height(node.right));
    int balance = getBalance(node);
    if (balance < -1 && key < node.left.key)    // LL
        return rightRotate(node);
    if (balance > 1 && key > node.right.key)    // RR
        return leftRotate(node);
    if (balance < -1 && key > node.left.key) {  // LR
        node.left = leftRotate(node.left);
        return rightRotate(node);
    }
    if (balance > 1 && key < node.right.key) {  // RL
        node.right = rightRotate(node.right);
        return leftRotate(node);
    }
    return node;
}
```

Después de insertar

Actualiza la altura del nodo y calcula su balance.

balance < −1

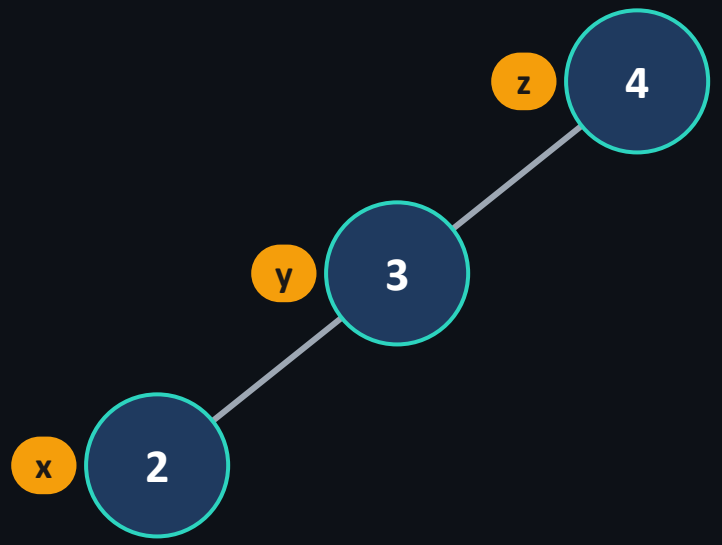
Pesa a la izquierda: caso LL o LR.

balance > 1

Pesa a la derecha: caso RR o RL.

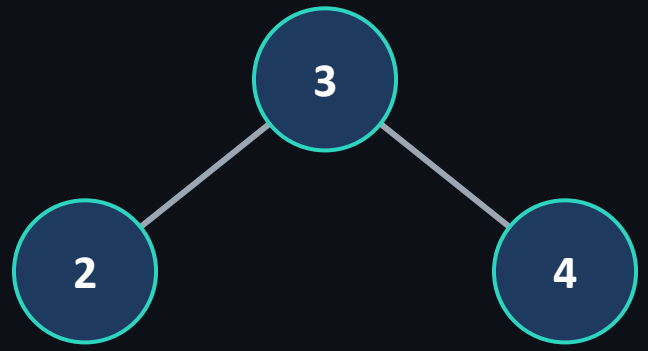
Eliminación en AVL (1/2): z = 4, caso LL

Se eliminan el 5 y el 8 (subárbol de z)



derecha en 4

Después de rotar a la derecha



Tras eliminar, se sube hasta el primer nodo desbalanceado (4, con factor −2); **y** y **x** son los hijos de mayor altura.

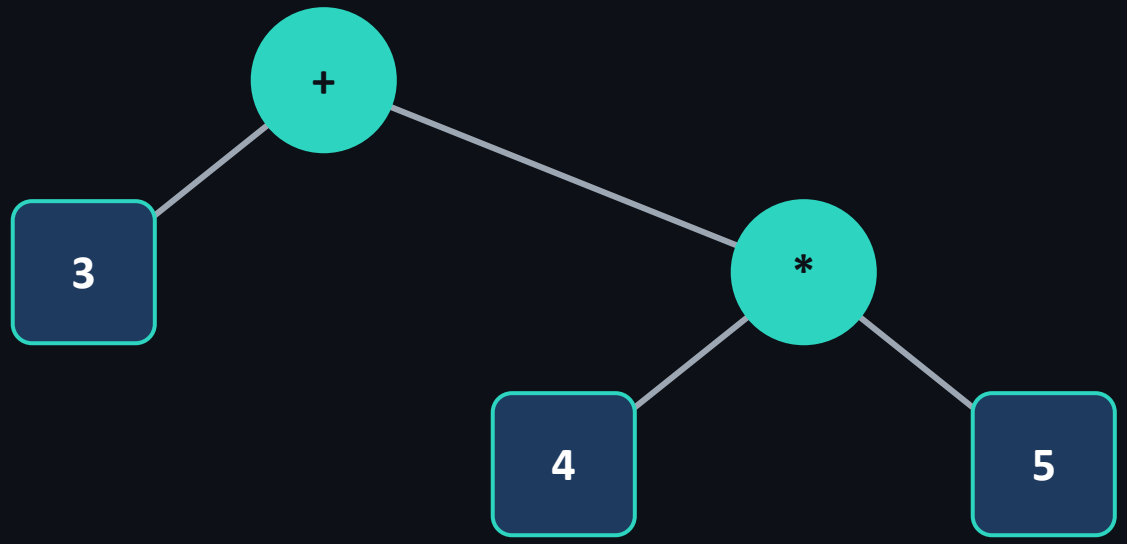
Eliminación en AVL (2/2): z = 10, caso RR



El 10 queda con factor **+2** (su derecha es más alta). **y** = 15 y **x** = 20 son hijos derechos: caso **RR**.

Árboles de expresión

Operador Operando



Hojas: operandos

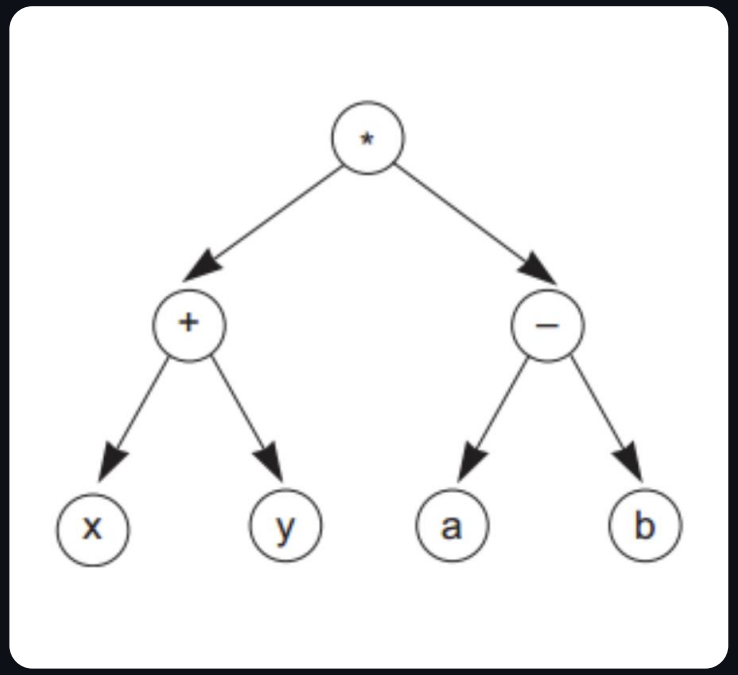
Números o variables: 3, 4 y 5.

Nodos internos: operadores

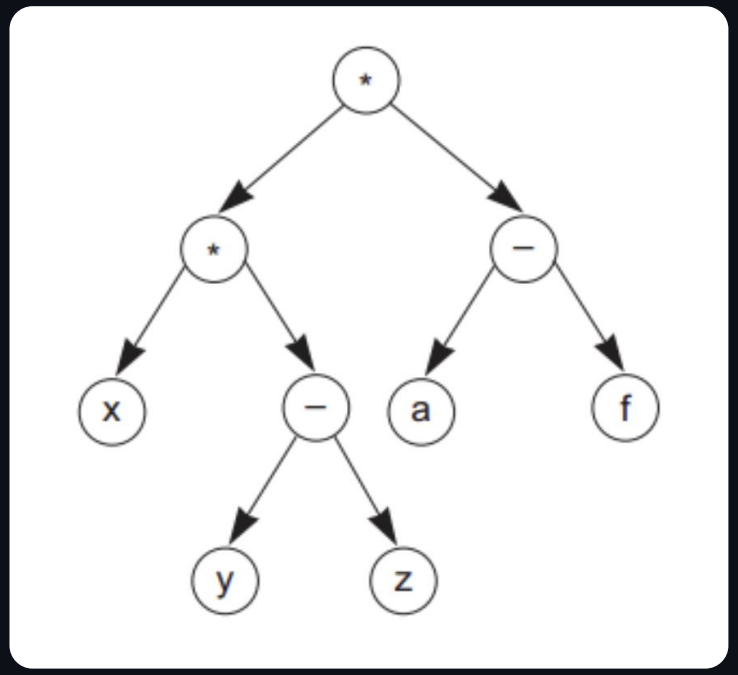
+ y * combinan a sus dos hijos.

Este árbol representa `3 + (4 * 5)`. También se usan para expresiones sintácticas de un lenguaje en la **compilación**.

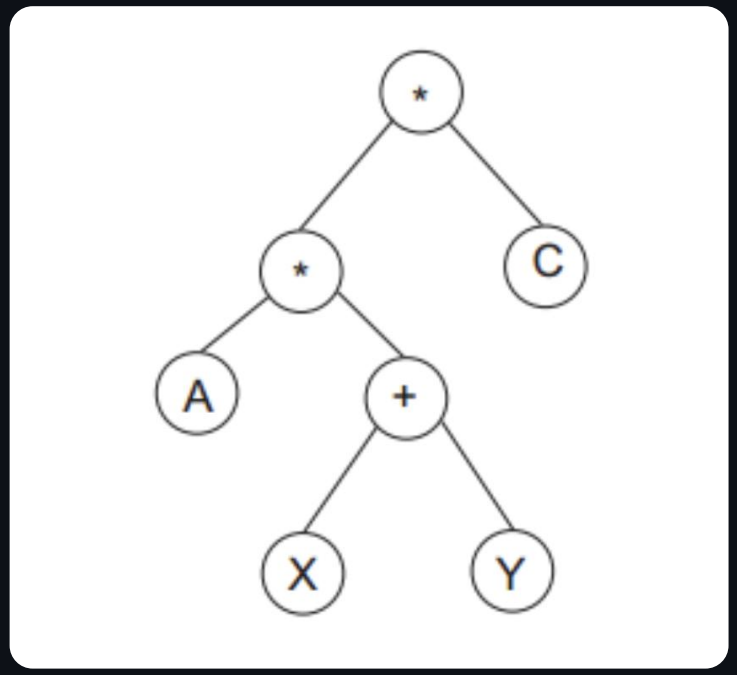
Del árbol a la notación infija: inorden



`(x + y) * (a - b)`



`(x * (y - z)) * (a - f)`



`((A * (X + Y)) * C)`

> Recorrer el árbol en **inorden** (izquierda, raíz, derecha) deja cada operador entre sus operandos.

De una expresión al árbol de expresión

1

A postfija

Con el algoritmo **Shunting Yard** (Dijkstra, 1961), en tiempo lineal.

2

Pila de árboles

Un operando es un árbol de un nodo; un operador une los dos árboles de la cima.

3

Árbol final

Al terminar queda un único árbol de expresión.

En la notación **infija** el operador va entre los operandos (`a + b`); en la **postfija**, después de ellos (`a b +`).

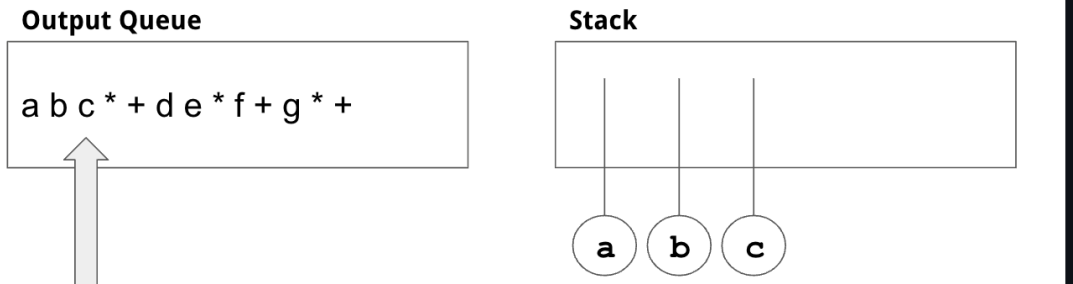
Shunting Yard: de infija a postfija

Se ha leído	Cola de salida	Pila de operadores
(a + (b * c	a b c	(+ (*
(a + (b * c)	a b c *	(+
(a + (b * c))	a b c * +	(vacía)
… + (((d * e	a b c * + d e	+ (((*
… + (((d * e)	a b c * + d e *	+ ((
toda la expresión	a b c * + d e * f + g * +	(vacía)

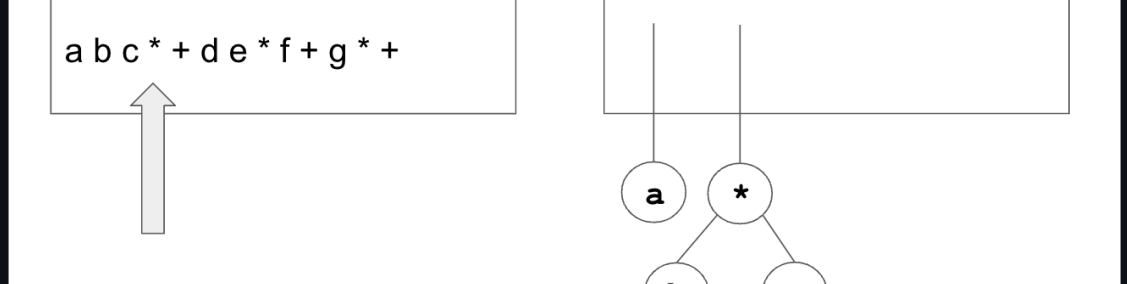
Expresión: `(a + (b * c)) + (((d * e) + f) * g)`

Reglas: un operando va a la cola; un operador se apila (antes se desapilan los de mayor precedencia).

Construir el árbol desde la postfija (1/2)



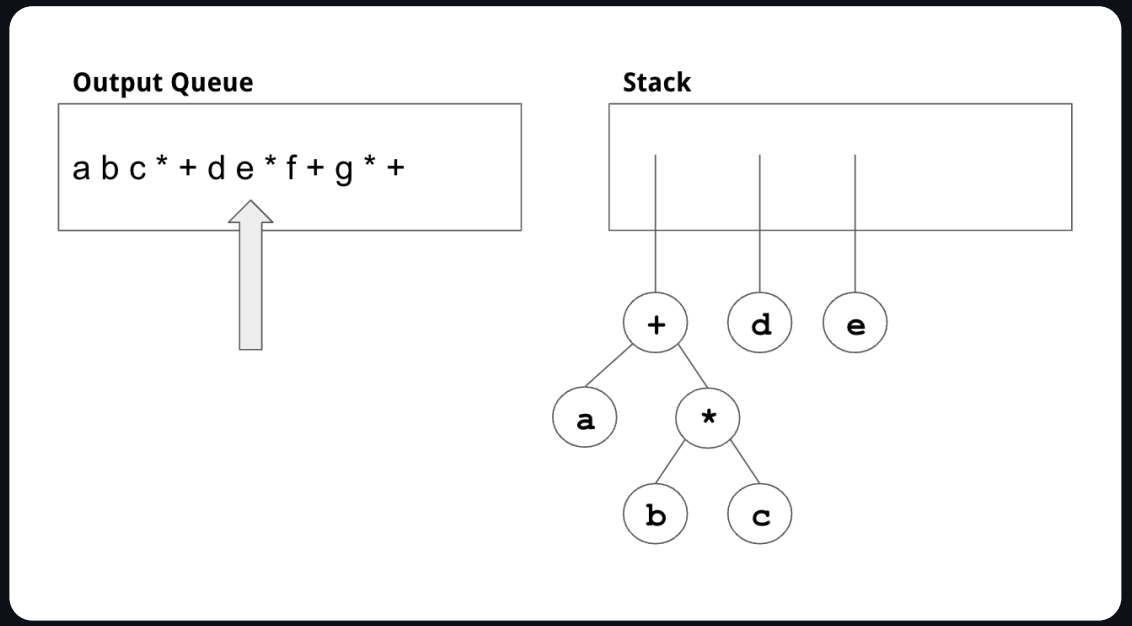
Se leen **a**, **b** y **c**: tres árboles de un nodo



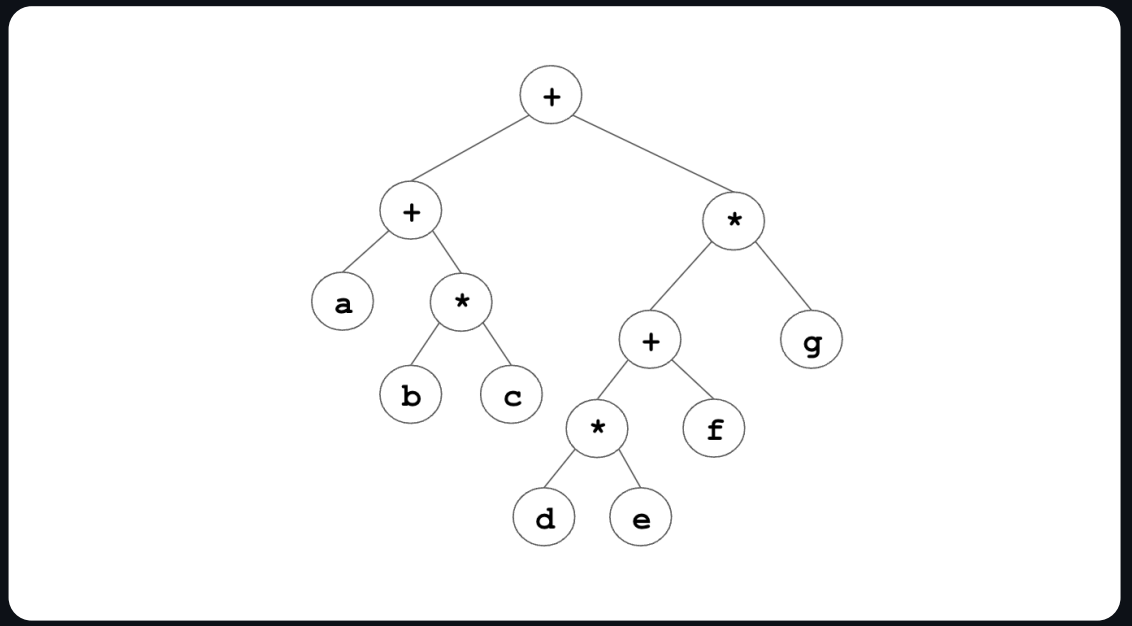
Se lee *****: une **b** y **c** bajo un nodo nuevo

> Cada **operando** se apila como un árbol de un nodo; cada **operador** desapila dos árboles y apila el nuevo.

Construir el árbol desde la postfija (2/2)



Tras +, d y e: tres árboles en la pila

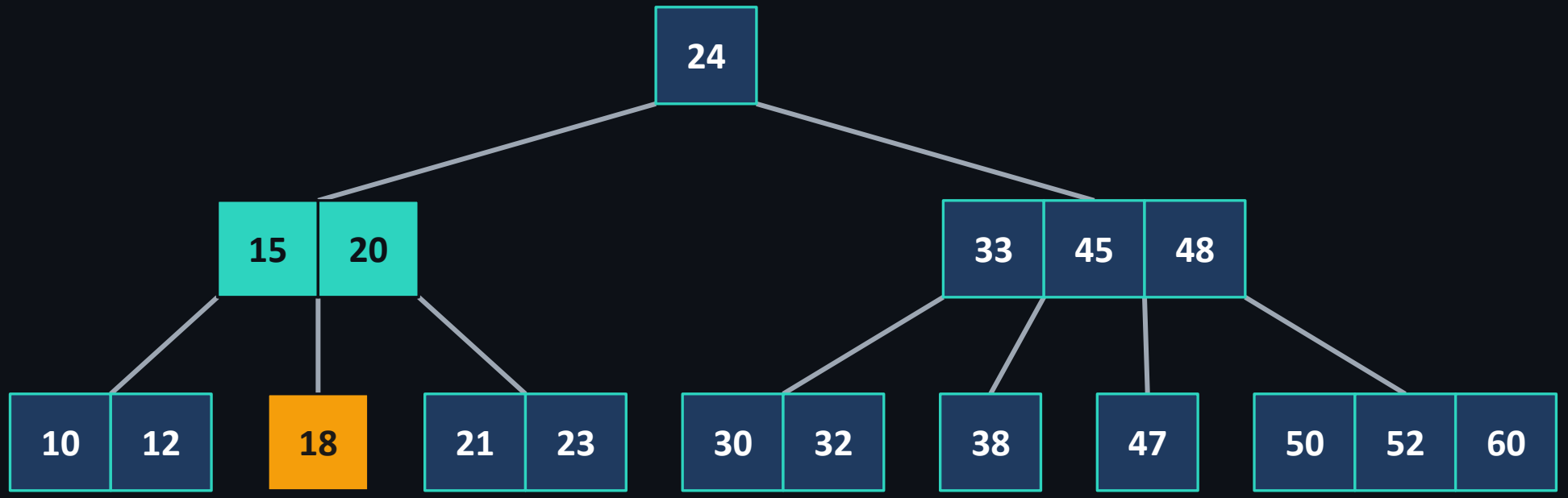


Árbol final de la expresión

> La raíz es el **último operador** de la postfija. El recorrido inorden (con paréntesis) devuelve la expresión infija.

Árboles B: árboles n-arios balanceados

Para grandes cantidades de datos en disco: bases de datos y sistemas de archivos (Bayer y McCreight, 1972).



Entre las llaves **15** y **20** hay un hijo cuyas llaves son mayores que 15 y menores que 20.

Características de un árbol B de orden m

Llaves y ramas

Las llaves ordenadas dividen las ramas en el orden esperado.

Hojas al mismo nivel

Todas las hojas están a la misma profundidad.

Orden m

Se define según el tamaño del bloque de disco.

Mínimo de llaves

Cada nodo (menos la raíz) guarda un mínimo, ≈ m/2; la raíz, al menos una.

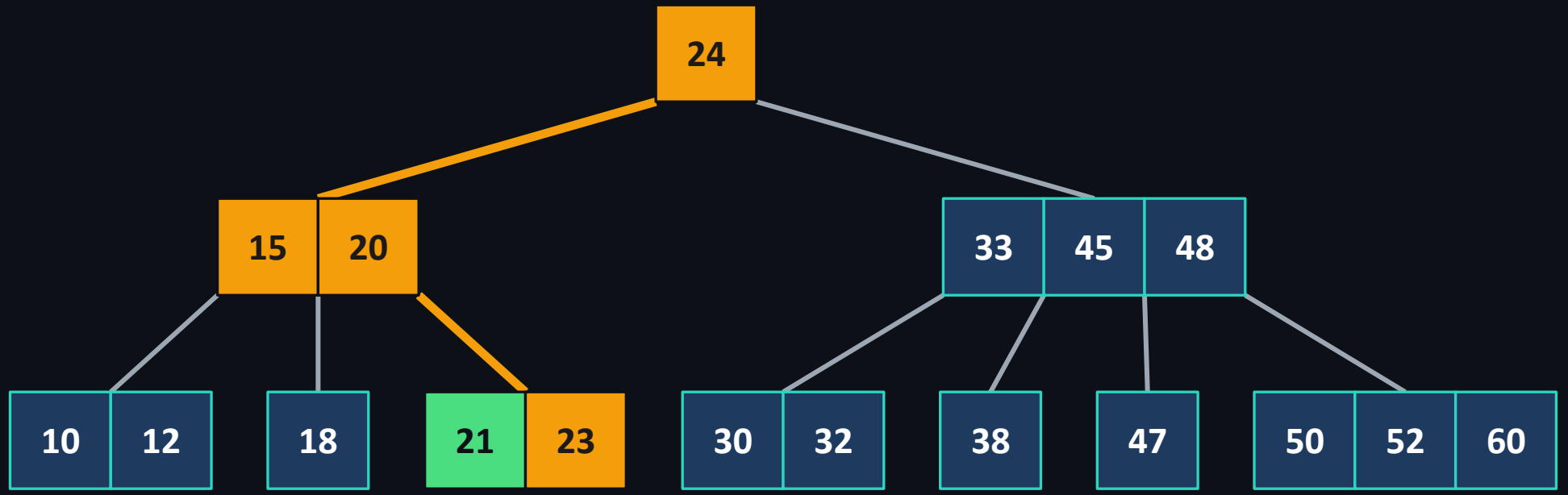
Inserción en hojas

La inserción siempre ocurre en una hoja.

Crecen desde la raíz

El árbol crece o decrece por la raíz.

Búsqueda en un árbol B: la llave 21



21 < 24: rama izquierda · 21 > 20 (última llave): última rama · en la hoja se encuentra el 21.

Código: estructura y búsqueda

Estructura básica

```
class BTreeNode {
    int order;
    int[] keys;
    int keyCount;
    BTreeNode[] branches;
}
```

Búsqueda de una llave

```
private Node Search(Node x, int key) {
    int i = 0;
    if (x == null) return x;
    for (i = 0; i < x.n; i++) {
        if (key < x.key[i]) break;
        if (key == x.key[i]) return x;
    }
    if (x.leaf) return null;
    else return Search(x.child[i], key);
}
```

Llaves y ramas

Un nodo guarda sus llaves ordenadas y las referencias a sus ramas.

Inserción en un árbol B

1

Buscar la hoja

Se desciende comparando con las llaves hasta llegar a una hoja.

2

Hoja con espacio

Se inserta la llave en su posición ordenada.

3

Hoja llena: dividir

La llave **mediana** sube al padre y las demás se reparten en dos nodos nuevos.

La división se repite hacia arriba hasta la raíz: el árbol B **crece desde la raíz**.

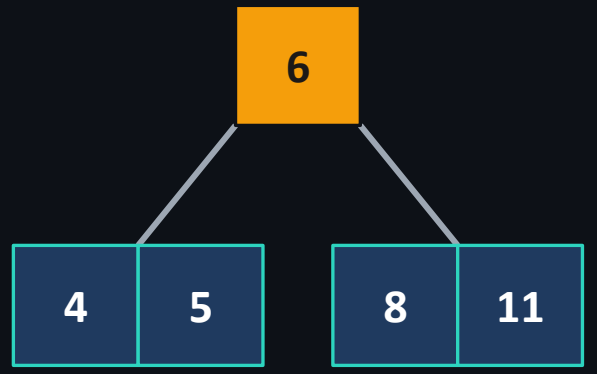
Inserción: dividir un nodo lleno (orden 5)

La raíz está llena



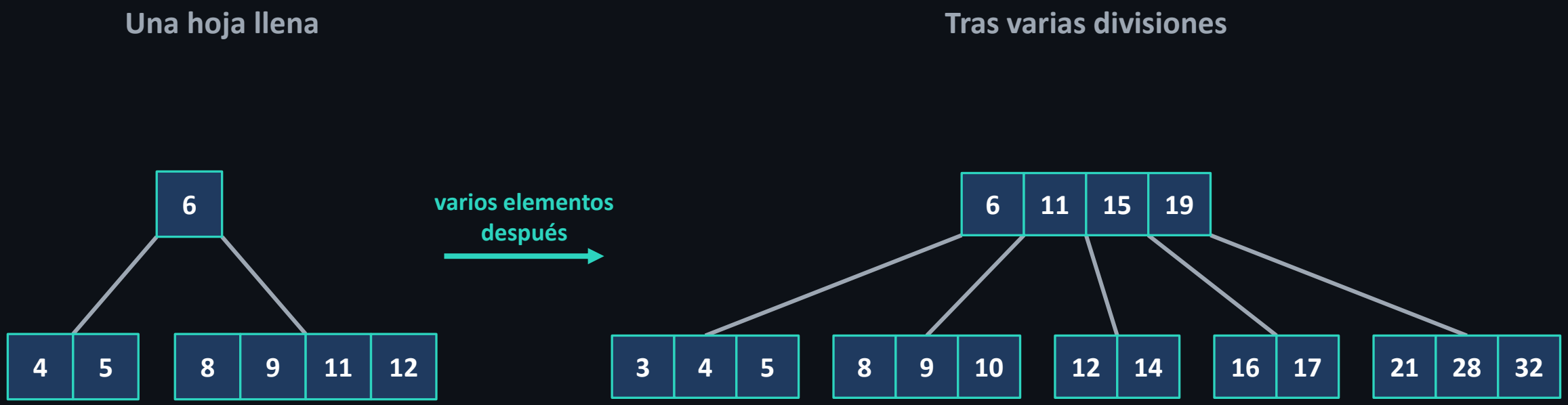
insertar 8

Se divide: la mediana sube



Orden 5: máximo 4 llaves por nodo. Con el 8 serían 5: la mediana (**6**) pasa a ser la nueva raíz.

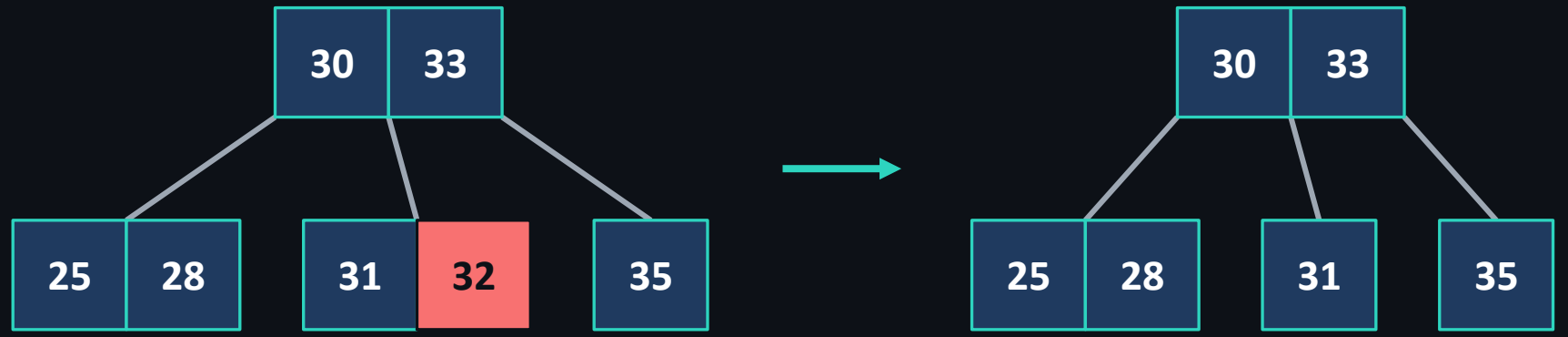
Inserción: el árbol crece hacia arriba



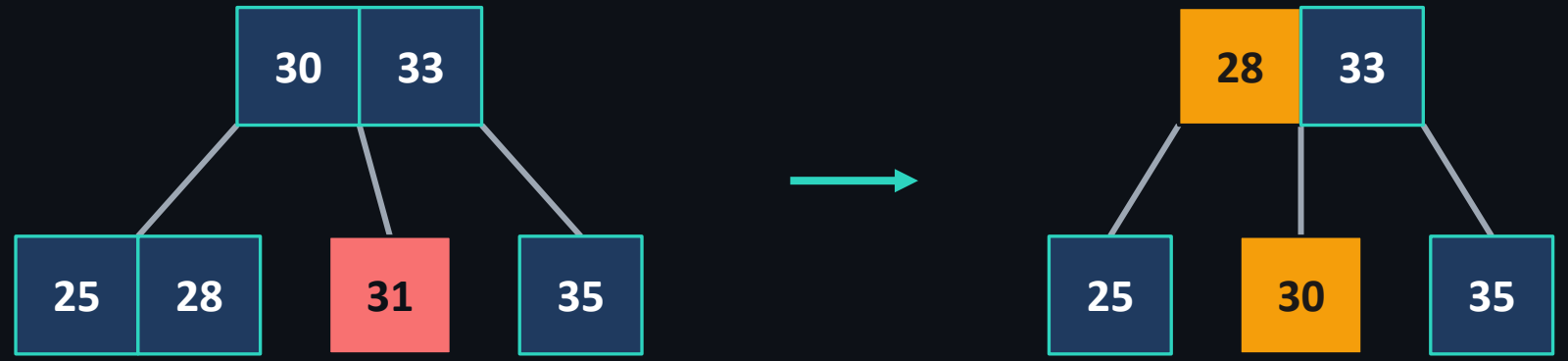
Cada vez que un nodo se llena se divide; las llaves medianas suben y las hojas siguen al mismo nivel.

Eliminación en una hoja (orden 3)

1.1 Hoja con más del mínimo: se elimina el 32



1.2 Hoja en el mínimo: pide prestado al hermano izquierdo (se elimina el 31)



Orden 3: mínimo de **1 llave** por nodo. Prestar: sube la mayor del hermano (28) y baja la del padre (30).

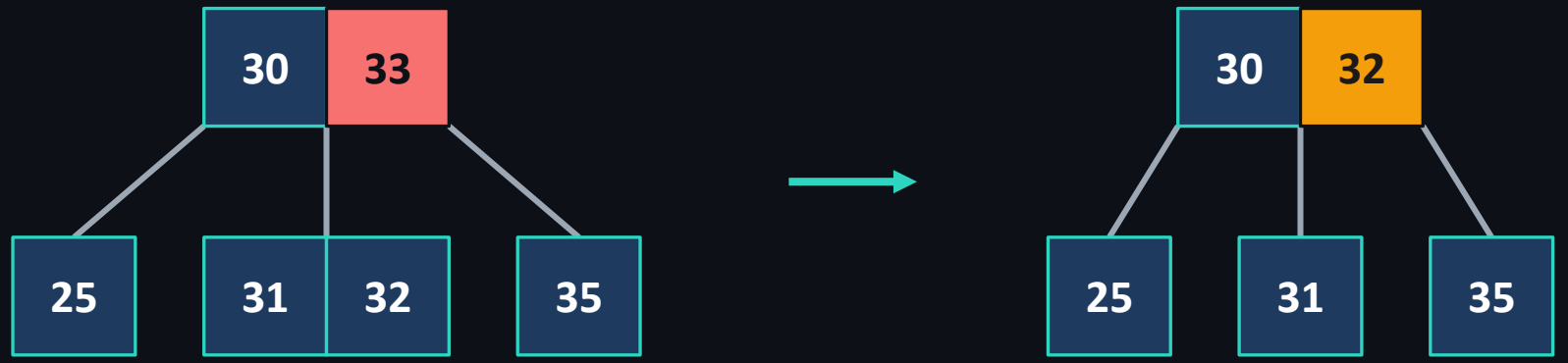
Eliminación en una hoja: fusionar



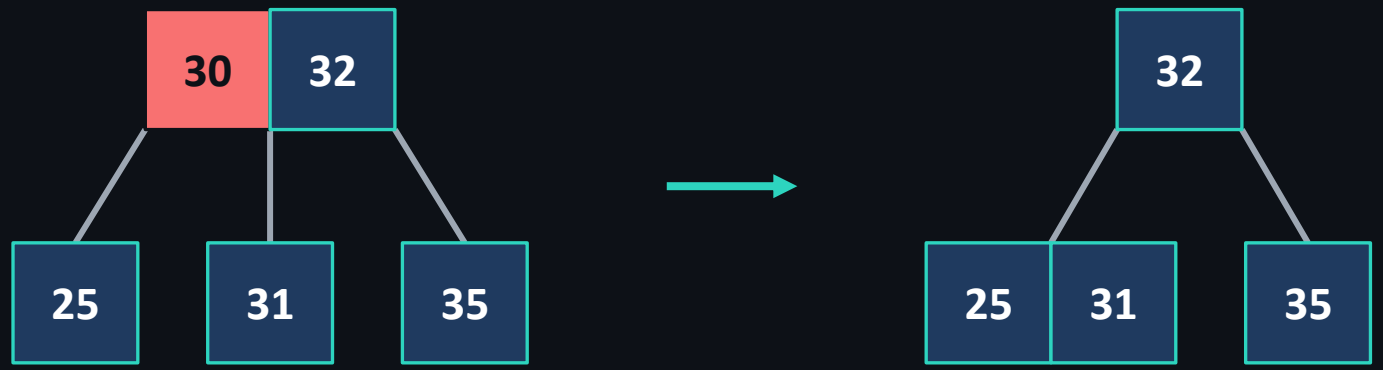
Si los hermanos también están en el mínimo, se **fusionan** con la llave del padre que los separa (28 baja).

Eliminación en un nodo interno

2.1 Se elimina el 33: lo reemplaza su predecesor (32)



2.2 Se elimina el 30: sus hijos en el mínimo se fusionan



2.1: el hijo vecino tiene más del mínimo. **2.2:** ninguno lo tiene: se fusionan los hijos.

Caso 3: se unen los hijos con el padre



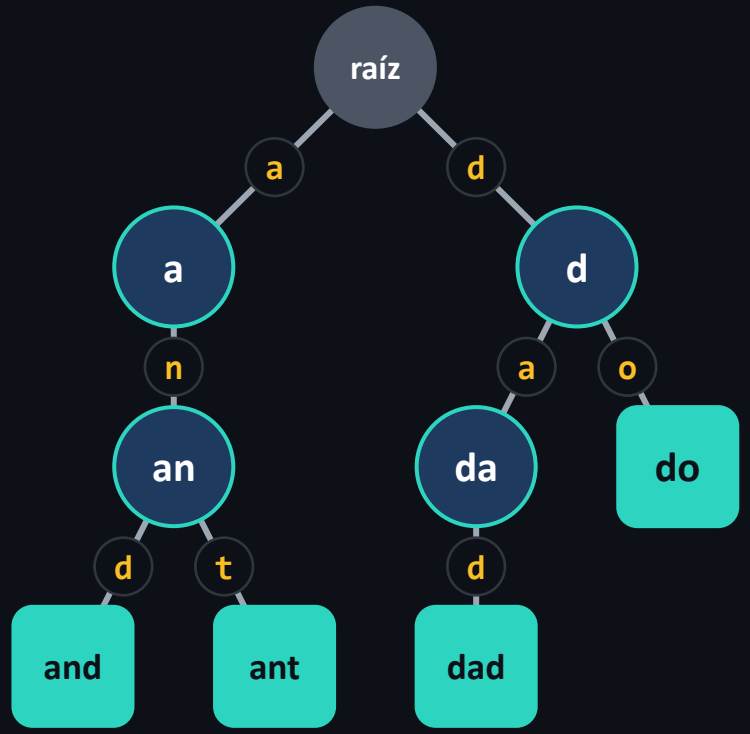
5 y 15 se fusionan; como 35 también está en el mínimo, el 20 baja y se une con él. Las hojas siguen al mismo nivel.

// tries

Trie: un árbol de prefijos

Árbol n-ario que guarda cadenas: cada rama es un carácter. Se pronuncia como el inglés «try».

Raíz (sin valor) Fin de palabra (EndOfWord)



Palabras guardadas

and, ant, dad y do.

Para qué sirve

Diccionarios y motores de búsqueda.

El TDA Trie y su estructura en Java

Nodo del trie

```
class TrieNode {
    TrieNode[] children = new TrieNode[ALPHABET_SIZE];
    boolean isEndOfWord;

    TrieNode() {
        isEndOfWord = false;
        for (int i = 0; i < ALPHABET_SIZE; i++)
            children[i] = null;
    }
}

class Trie {
    TrieNode root;
}
```

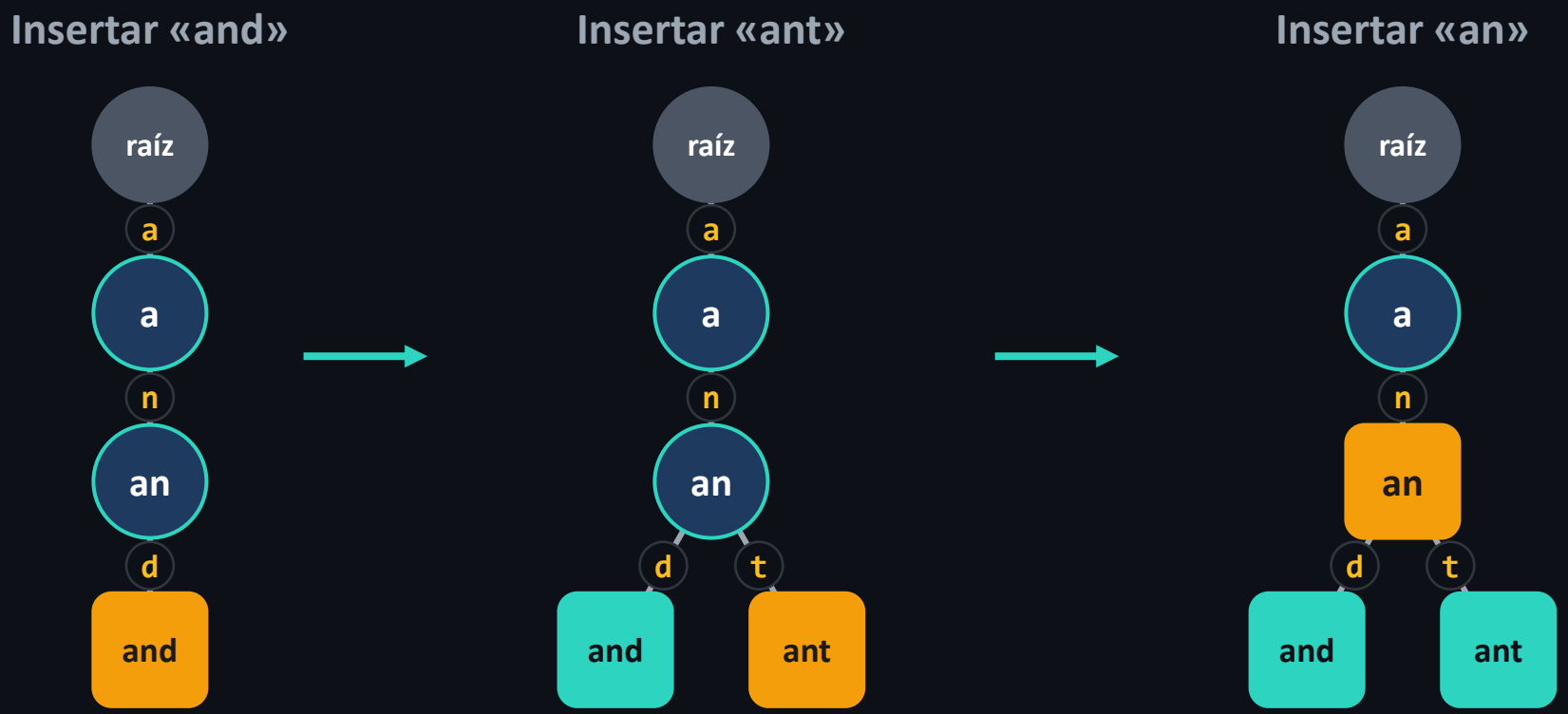
Operaciones

insert, **search** y **delete** de cadenas de caracteres.

Arreglo de hijos

El carácter funciona como índice: `'a'` es la posición 0.

Inserción en un trie



Se reutilizan los nodos que ya existen. Si la llave es un **prefijo** de otra, solo se marca su último nodo como fin de palabra.

Código: insertar en un trie

```
void insert(String key) {
    int level;
    int length = key.length();
    int index;
    TrieNode pCrawl = root;

    for (level = 0; level < length; level++) {
        index = key.charAt(level) - 'a';
        if (pCrawl.children[index] == null)
            pCrawl.children[index] = new TrieNode();

        pCrawl = pCrawl.children[index];
    }
    pCrawl.isEndOfWord = true;  // último nodo
}
```

Un nodo por carácter

Si el hijo no existe, se crea; luego se desciende por él.

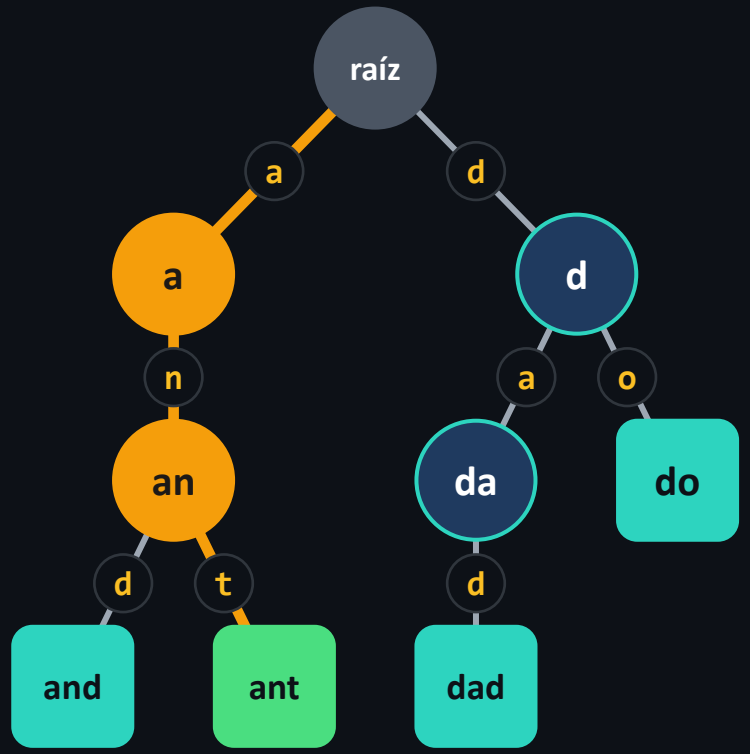
Fin de palabra

El último nodo se marca con `isEndOfWord = true`.

Profundidad

La longitud de la llave define la profundidad.

Búsqueda en un trie



Consulta	Resultado
ant	true: llega a «ant», que es fin de palabra
an	false: el nodo existe, pero no es fin de palabra
dot	false: no hay rama «t» bajo «do»

La búsqueda termina por **fin de la cadena** (se revisa `isEndOfWord`) o por **falta de rama** (la llave no existe).

Código: buscar en un trie

```
boolean search(String key) {
    int level;
    int length = key.length();
    int index;
    TrieNode pCrawl = root;

    for (level = 0; level < length; level++) {
        index = key.charAt(level) - 'a';

        if (pCrawl.children[index] == null)
            return false;

        pCrawl = pCrawl.children[index];
    }
    return pCrawl.isEndOfWord;
}
```

Falta una rama

Devuelve `false` sin revisar el resto de la llave.

Se agotó la llave

Devuelve el valor de `isEndOfWord` del último nodo.

// cierre

Comparación de las estructuras

Estructura	Idea clave	Implementación	Uso típico
BST	izquierdo < nodo < derecho	Nodos enlazados	Búsqueda eficiente
Heap	Completo; padre ≥ hijos (máx.)	Arreglo	Colas de prioridad, heap sort
AVL	BST con balance −1, 0 o 1	Nodos y rotaciones	Búsqueda en O(log n) garantizada
Expresión	Hoja: operando; nodo: operador	Nodos enlazados	Compiladores, evaluar expresiones
Árbol B	N-ario; hojas al mismo nivel	Llaves y ramas	Disco: bases de datos y archivos
Trie	Un carácter por nodo	Arreglo de hijos	Diccionarios, motores de búsqueda

Todas organizan los datos en jerarquías: la **forma** del árbol define qué operaciones son rápidas.

// cierre

Ahora usted

Aplique lo aprendido (las respuestas están en las notas del orador):

A · BST

Inserte 50, 30, 70, 20, 40, 60 y 80 en un BST vacío. Escriba el recorrido inorden.

B · AVL

Inserte 10, 20 y 30 en un AVL vacío. ¿Qué caso de rotación ocurre?

C · Heap

Inserte 5, 10, 3 y 8, en ese orden, en un heap máximo. ¿Cómo queda el arreglo?

D · Expresión

Dibuje el árbol de `(a + b) * c` y escriba su expresión postfija.

// cierre

Resumen

- **Árboles:** una raíz y nodos con cero o más hijos; insertar, eliminar, buscar y recorrer.
- **BST:** izquierdo < nodo < derecho; el inorden da el orden; puede degenerar en una lista.
- **Heap:** árbol completo con la propiedad de heap; se guarda en un arreglo.
- **AVL:** BST con factor de balance −1, 0 o 1; rotaciones LL, RR, LR y RL.
- **Árboles de expresión:** hojas = operandos, nodos internos = operadores.
- **Árboles B y tries:** árboles n-arios para datos en disco y para cadenas de caracteres.

Orígenes mencionados en el capítulo: AVL (Adelson-Velsky y Landis, 1962) · árboles B (Bayer y McCreight, 1972) · Shunting Yard (Dijkstra, 1961).